



Análise dos resultados do modelo BERTImbau ajustado para a tarefa de reconhecimento de entidades nomeadas

Analysis of the results of the BERTImbau model for the task of recognition of named entities

DOI: 10.55905/revconv.16n.12-107

Recebimento dos originais: 06/11/2023

Aceitação para publicação: 12/12/2023

Mateus Brugnaroto

Graduado em Engenharia da Computação
Instituição: Universidade Federal de Pelotas
Endereço: Pelotas – RS, Brasil
E-mail: mbrugnaroto@inf.ufpel.edu.br

Ulisses Brisolara Corrêa

Doutor em Ciência da Computação
Instituição: Universidade Federal de Pelotas
Endereço: Pelotas – RS, Brasil
E-mail: ub.correa@inf.ufpel.edu.br

Larissa Astrogildo de Freitas

Doutora em Ciência da Computação
Instituição: Universidade Federal de Pelotas
Endereço: Pelotas – RS, Brasil
E-mail: larissa@inf.ufpel.edu.br

RESUMO

Caracterizada por avanços tecnológicos críticos e pela emergência de técnicas de Inteligência Artificial (IA), a era da transformação digital proporciona uma tomada de decisão mais rápida e eficiente por meio da análise de dados, conhecimento e habilidades. No entanto, simplesmente coletar dados não gera, por si só, uma vantagem competitiva. É necessário desenvolver técnicas robustas e confiáveis que possam transformar os dados em informações úteis, tornando-se inteligência de mercado e inteligência operacional. Para isso, o modelo *Bidirectional Encoder Representations from Transformers* (BERT) para o português, chamado BERTImbau, alcançou um estado da arte na tarefa de Reconhecimento de Entidades Nomeadas (REN) em Processamento de Linguagem Natural (PLN), alcançando 83,93% de Medida-F. No entanto, métricas clássicas de Precisão, Revocação, Acurácia e Medida-F, podem não refletir completamente o desempenho do modelo, uma vez que, apenas validam a classificação ou a previsão do modelo sem considerar o contexto ao redor dos resultados. Com isso em mente, este trabalho analisa sistematicamente os resultados do modelo BERTImbau ajustado para a tarefa de REN. A análise visa verificar as fraquezas e os padrões de erro do modelo e confirmar se as métricas clássicas podem justificar o desempenho do modelo na tarefa de REN. Nossa conclusão é que o modelo fornece resultados confiáveis por meio de uma análise detalhada.



Palavras-chave: processamento da língua natural, reconhecimento de entidades nomeadas, BERTimbau.

ABSTRACT

The digital transformation era is characterized by critical technological advances and the emergence of Artificial Intelligence (AI) techniques, which provide faster and more efficient decision-making through data analysis, knowledge, and skills. However, simply collecting data does not generate a competitive advantage on its own. It's necessary to develop robust and reliable techniques that can transform data into useful information, becoming market intelligence and operational intelligence. To achieve this, the Bidirectional Encoder Representations from Transformers (BERT) model for Portuguese, known as BERTimbau, has reached a state-of-the-art level in the Natural Language Processing (NLP) task of Named Entity Recognition (NER), with a Measure-F of 83.93%. However, classic assessment tools such as Precision, Recall, Accuracy, and F-Measure classical metrics may not fully reflect the model's true performance, as they only validate the model's classification or prediction without considering the context around the results. With this in mind, this work systematically analyzes the results of the BERTimbau model adjusted for the NER task to verify the model's weaknesses and error patterns. The analysis aims to confirm whether classical metrics can justify the model's performance in the NER task. Our conclusion is that the model provides reliable results through a fine-grained analysis.

Keywords: natural language processing, named entity recognition, BERTimbau.

1 INTRODUÇÃO

Com a chegada da era da Transformação Digital e, consequentemente, com o crescimento exponencial do volume de dados, organizações de todo o mundo vêm se interessando pela análise de seus dados (Henke et al., 2016). Sendo esse um dos pilares para o avanço na área de análise de dados, técnicas de extração de informação de dados são aprimoradas constantemente por grupos de pesquisa em instituições de ensino, em projetos *open-source* e empresas privadas.

Através da análise de dados, empresas conseguem validar suas ideias, suas movimentações no mercado, seus projetos, seus investimentos em recursos (por exemplo: tempo, dinheiro, etc.), entre vários outros aprimoramentos no planejamento estratégico. Porém, para isso ocorrer, é necessário desenvolver técnicas assertivas que proporcionem coletar insights confiáveis (Tukey, 1962). Pensando nisso e nos resultados alcançados apresentados na literatura, diversos trabalhos envolvem o uso de Aprendizagem Profunda (do inglês, *Deep Learning*) para proporcionar esses resultados aos gestores de organizações (da Silva Magalhães Gomes & Evsukoff, 2019).



Sabendo disso e com o aumento de dados produzidos periodicamente através sistemas eletrônicos, páginas web em geral, documentos empresariais, entre outros, organizações buscam vantagem de mercado através de dados do tipo textual (Lopes, 2004). De tal modo, muito por conta dos avanços computacionais e o surgimento de técnicas de Inteligência Artificial, através de técnicas de Processamento de Língua Natural (PLN) está sendo possível desenvolver modelos computacionais para realização de tarefas que dependem de informações expressas em alguma língua natural, como, por exemplo, o português (do Lago Pereira, 2011).

Os primeiros sistemas de PLN eram modelados para detectar padrões em texto baseados em regras gramaticais. Com o avanço do poder computacional e o uso de Aprendizado de Máquina, estes sistemas começaram a ser migrados para modelos estatísticos altamente escaláveis (Souza et al., 2019). Dessa forma, tornou-se possível ampliar as tarefas de PLN, abrangendo hoje tarefas como Reconhecimento de Entidades Nomeadas, Análise de Sentimentos, Tradução Automática, Análise do Discurso, entre outras (Nadkarni et al., 2011).

A tarefa de PLN conhecida como Reconhecimento de Entidades Nomeadas (REN) tem demonstrado ser muito importante por conta de sua capacidade de categorizar Entidades Nomeadas (EN), em categorias pré-definidas, tais como: Pessoa, Local, Organização e Tempo (Nadeau & Sekine, 2007).

Além de sua importância como uma tarefa fim, a tarefa de REN também é utilizada como uma etapa de pré-processamento em outras tarefas de PLN, ou seja, como uma tarefa meio. Um exemplo de sua utilização é na modelagem sistemas de identificação de relacionamento entre entidades, que permitem assimilar atributos a uma determinada Pessoa/Organização em um texto. Apesar de parecer uma tarefa simples, o REN é altamente sensível à semântica e ao ambiente textual. Isto é, a ambiguidade e a imprecisão podem classificar o mesmo nome próprio em categorias distintas (por exemplo a palavra “Brasil”, dependendo do contexto, pode ser interpretada como um Local ou uma Organização). Além disso, existem diversas classificações para ENs, as quais possuem diferentes critérios de avaliação. Por conta disso, o método de avaliação se torna ainda mais prejudicado na hora de realizar a classificação das ENs (Souza et al., 2019).

Em 2018, a Google desenvolveu uma abordagem de pré-treinamento chamada de BERT (do inglês, *Bidirectional Encoder Representations from Transformers*), com o objetivo de aprimorar os resultados das tarefas de PLN através do conceito de bidirecionalidade. Este modelo



alcançou o desempenho de estado-da-arte em 11 tarefas de PLN para a língua inglesa. Através dessa abordagem, utilizando pré-treinamento e *fine-tuning*, não é necessário treinar e desenvolver arquiteturas específicas para cada tarefa de PLN (Devlin et al., 2019).

Segundo Devlin et al. (2019), um dos motivos do modelo BERT possuir um desempenho superior a outros modelos em tarefas de PLN é por ser a primeira representação de linguagem profundamente bidirecional e não supervisionada. Eles afirmam que os modelos atuais, principalmente por serem unidirecionais, restringem o poder das representações pré-treinadas, especialmente em abordagens de *fine-tuning*. Ou seja, tarefas onde é fundamental incorporar o contexto de ambas as direções, como no nível de token (REN é um caso), modelos unidirecionais não conseguem atingir os melhores resultados. Tal depreciação é aliviada, segundo eles, pela bidirecionalidade do modelo BERT.

De modo a abranger outros idiomas, além do inglês, o modelo BERT multilinguagem, conhecido como mBERT, foi desenvolvido para os 102 idiomas com maior quantidade de dados na Wikipédia. Pré-treinado em vários idiomas, utilizando corpus baseado nos dados da Wikipédia, o modelo BERT pode ser ajustado em um idioma e avaliado em outro idioma (processo conhecido como aprendizado zero-shot) (Devlin et al., 2019).

Contudo, esse modelo obteve uma queda no desempenho comparado ao modelo original. Ao aprender estruturas semânticas universais por conta do aprendizado zero-shot, o modelo ignora as diferenças específicas de cada idioma (Devlin et al., 2019). Em razão disso, algumas instituições desenvolveram modelos específicos para uma única língua a partir do modelo *open-source* disponível.

Dessa forma, modelos BERT dedicados a uma única língua foram desenvolvidos para melhorar os resultados obtidos pelo mBERT. Em 2019, a empresa NeuralMind-AI desenvolveu o primeiro modelo baseado na abordagem BERT para o português, denominado BERTimbau, o qual atingiu o estado da arte na tarefa de REN em português (Souza et al., 2019).

Mesmo atingindo o estado-da-arte em diversas tarefas de PLN, o modelo BERT apresentou algumas limitações no desempenho que o impediram de garantir ótimos resultados. Portanto, é fundamental entender os pontos que estão diretamente ligados ao seu desempenho quando aplicado a uma tarefa de PLN em específico.

As avaliações de desempenho de modelos que atuam a tarefa de classificação são, geralmente, baseadas em um conjunto de métricas clássicas, como Precisão, Revocação,



Acurácia e Medida-F. No entanto, essas métricas possuem limitações, apresentando uma visão geral do desempenho dos modelos, mas dificultando a análise detalhada dos erros.

De tal forma, há pelo menos dois pontos que mostram o quão frágil e ineficiente essas métricas são no momento de avaliar o modelo e, conseqüentemente, indicar a causa dos seus acertos e de suas falhas, que são: (1) Não é possível, através dessas métricas, identificar com clareza os pontos que estão influenciando diretamente no resultado do modelo. Essas métricas não nos fornecem os motivos pelo qual o modelo está anotando um determinado dado como verdadeiro positivo ou falso positivo, por exemplo. Ou seja, por exemplo, não é possível provar que o modelo está realmente aprendendo, tendo em vista que ele possa estar simplesmente chutando suas anotações. (2) Outro ponto importante em relação a essas métricas remete a complexidade em melhorar um sistema em que não se conhece os problemas. É muito improvável e difícil que um engenheiro ou um cientista consiga melhorar um modelo sem ao menos saber aonde está a sua falha. Visto desse modo, através dessas métricas, tais informações não são extraídas com exatidão.

Para minimizar isso, existem diferentes metodologias para extrair melhor a avaliação sobre os resultados de um modelo. Dentre elas pode-se citar as análises de erro de baixa granularidade de sistemas e a avaliação de interpretabilidade (Fu et al., 2020).

Pensando nisso, este trabalho visa realizar uma análise de erro de baixa granularidade nos resultados do modelo BERTimbau, quando aplicado a tarefa de REN. O intuito, além de compreender os resultados do modelo, é verificar seus pontos fracos, seus padrões de erros e confirmar se o desempenho apresentado na tarefa REN é, de fato, justificado.

O restante do trabalho está organizado da seguinte forma. Na seção 2 é descrito o Referencial Teórico, Tecnológico e Trabalho Relacionado; na seção 3 é apresentada a Metodologia; na seção 4 são apresentados a Discussão dos Resultados; e por fim, na seção 5 são discutidas as Considerações Finais e os Trabalhos Futuros.

2 REFERENCIAL TEÓRICO, TECNOLÓGICO E TRABALHO RELACIONADO

Nesta seção são apresentados os principais conceitos para o entendimento deste trabalho: REN, HAREM, Transformers, BERT, BERTimbau, Avaliação de Desempenho e Trabalho Relacionado.



2.1 RECONHECIMENTO DE ENTIDADES NOMEADAS

O REN é uma tarefa importante no PLN por ser capaz de identificar e classificar ENs em categorias predefinidas. Por mais que existam diferentes definições para ENs, podemos compreendê-las como nomes próprios ou designadores rígidos expressos em textos (Nadeau & Sekine, 2007).

Os tipos mais comuns de entidades encontrados em textos são: substantivos próprios, como nomes de pessoas e nomes de organizações; temporais, como hora, dia, mês, ano e tempo; numéricos, como porcentagens e métricas.

Do mesmo modo, categorias de entidades específicas de domínio também podem ser definidas. Em trabalhos na área geológica, por exemplo, podemos definir “mineral”, “rocha” e “pedra” como algumas das possíveis categorias de entidades (Nadeau & Sekine, 2007).

A ideia por trás da importância dessa tarefa é que ENs são elementos importantes para a compreensão de documentos de texto. Através das ENs, são fornecidos subsídios fundamentais para outras tarefas de PLN, como exemplo de respostas automáticas do tipo: “sobre quem se trata essa notícia?”, “sobre quais organizações estão se tratando?”, entre outros (da Silva, 2012).

2.2 HAREM

A Avaliação de Sistemas de Reconhecimento de Entidades Mencionadas (HAREM), é um evento de avaliação conjunta da língua portuguesa na área de REN. O objetivo do HAREM estava voltado em promover o desenvolvimento de sistemas de RENs, como também propor uma metodologia diferente de avaliação e que abrangesse as especificidades da tarefa. Foram realizados duas edições do evento, das quais foram compilados os corpus chamados de Coleções Douradas (CD), Primeiro HAREM, Mini HAREM e Segundo HAREM (Santos & Cardoso, 2007).

A primeira edição do HAREM foi dividida em duas partes: Primeiro HAREM e Mini HA-REM. Para essa edição foi construído um corpus contendo textos de diferentes gêneros textuais e fontes. Com a ideia de modelar coleções devidamente anotadas (chamadas posteriormente de Coleções Douradas) e que consistiriam em um padrão de diretivas, o conteúdo das coleções variaram desde textos retirados da web, de entrevistas, e-mails, até textos jornalísticos, políticos, literários, técnicos e expositivos (Santos & Cardoso, 2007).



A primeira etapa ocorreu em 2005, envolvendo 10 equipes participantes de seis países diferentes. Para a anotação contida na CD do Primeiro HAREM, a coleção foi dividida em conjuntos de texto e entregue para cada um dos 10 participantes. Posteriormente, cada conjunto foi revisto por anotadores independentes e outros participantes (Santos & Cardoso, 2007). No total, foram anotados 5.110 ENs distribuídas entre as 10 categorias, sendo elas: Abstração, Acontecimento, Coisa, Local, Obra, Pessoa, Organização, Tempo, Valor e Variado (e Silva, 2020).

A segunda etapa ocorreu em 2006, envolvendo 5 das equipes participantes do Primeiro HAREM. Uma nova CD foi anotada para o Mini HA-REM utilizando o mesmo processo, porém distribuindo os conjuntos da coleção entre as 5 equipes. Contendo 128 documentos, foram anotadas 3.758 ENs, distribuídas entre as mesmas categorias do Primeiro HAREM (e Silva, 2020).

A segunda edição do HAREM foi realizada em 2008 e reuniu dez novas equipes participantes. Para a anotação da CD do Segundo HAREM, foram extraídos 129 documentos do português do Brasil e de Portugal, totalizando 7.272 ENs distribuídas em 10 categorias, sendo elas: Abstração, Acontecimento, Coisa, Local, Obra, Pessoa, Organização, Tempo, Valor e Outro (e Silva, 2020).

Duas das diferenças entre a primeira e a segunda edição foram a inserção de textos de domínios ausentes, como artigos da Wikipédia, e a substituição da categoria “Variado” por “Outro”. Essa alteração na categoria foi, posteriormente, corrigida nos CDs anteriores (Mota & Santos, 2008).

2.3 TRANSFORMERS

O *Transformer* é um modelo de transdução (conversão de sequências de entrada em sequências de saída) baseado unicamente em *Attention Mechanism* para computar representações de sua entrada e saída sem usar Redes Neurais Recorrentes alinhados por sequência ou convolução. Possui uma estrutura codificador-decodificador, onde o codificador realiza o mapeamento de uma sequência de entrada para uma sequência de representações contínuas e o decodificador, dado a sequência de representações contínuas geradas pelo codificador, produz uma previsão para uma tarefa de PLN específica. A estrutura do *Transformer* usa camadas de



Self-attention empilhadas e totalmente conectadas no codificador e decodificador (Vaswani et al., 2017).

Diferente dos modelos direcionais, que leem a entrada de texto sequencialmente, ou seja, da esquerda para a direita ou da direita para a esquerda, o codificador *Transformer* lê toda a sequência de palavras de uma vez. De tal forma, permite que o modelo aprenda o contexto de uma palavra com base nas sentenças a esquerda e direita de uma palavra e não somente baseada em dicionário ou em uma única direção da sentença (Vaswani et al., 2017).

Para entender melhor, utilizamos três exemplos simples, respectivamente, para representar palavras sem condicionar contexto a elas, palavras condicionadas ao contexto unidirecional e palavras condicionadas ao contexto bidirecional: (1) Em frases onde a palavra não é condicionada ao contexto, como “A agência do banco não estava disponível naquele momento” e “Eu vou me sentar no banco”, a palavra “banco” será representada do mesmo modo em ambas as frases. (2) Em uma frase onde a palavra é condicionada ao contexto unidirecional, como “Eu tive acesso ao banco digital e acessei minha conta bancária”, a palavra “banco” será representada, por exemplo, com base na sentença a sua esquerda “Tive acesso ao”. Contudo, como não será condicionada ao contexto a sua direita também, a sua representação pode divergir do seu real significado. (3) Já onde a palavra é condicionada pelo contexto bidirecional, na frase “Eu tive acesso ao banco digital e acessei minha conta bancária”, a palavra “banco” será representada utilizando os dois contextos: “Tive acesso ao” e “digital e entrei em minha conta bancária”. Assim, o processo garante uma assertividade maior ao representar a palavra.

2.3.1 *Attention Mechanism*

Attention Mechanism observa uma sequência de entrada e decide em cada posição quais partes da sequência são importantes. Pode ser descrito como um mapeamento de vetores de consulta e de conjuntos de pares de valores-chave, onde a saída é calculada através da soma ponderada dos valores e o peso atribuído a cada valor é calculado por uma função de compatibilidade dos valores do vetor de consulta com os valores do vetor de chave correspondente (Vaswani et al., 2017).



2.3.2 *Self-attention*

Self-attention é um mecanismo de atenção que realiza o relacionamento de diferentes posições de uma única sequência com o intuito de computar uma representação da sequência. Ou seja, permite que o sistema observe outras palavras em uma frase para obter um entendimento mais preciso de uma determinada palavra na frase (Vaswani et al., 2017).

2.3.3 *Attention-heads*

Attention-heads é o mecanismo de calcular várias vezes o *Self-attention* na arquitetura do *Transformer*. Os cálculos do *Self-attention* são realizados em paralelo e independentemente, mas por fim, suas saídas são concatenadas e transformadas linearmente. Isso permite o mecanismo atender conjuntamente às informações de diferentes subespaços de representação em diferentes posições (Vaswani et al., 2017).

2.4 BERT

BERT é um modelo de representação de linguagem baseado em *Transformer Encoders*. O modelo é projetado para pré-treinar representações bidirecionais profundas de texto não rotulado, condicionando conjuntamente o contexto esquerdo e direito das palavras do texto (Devlin et al., 2019). Como o objetivo do BERT é gerar um modelo de linguagem, apenas o mecanismo do codificador da arquitetura do *Transformer* é utilizado. De tal forma, a arquitetura da rede neural do modelo BERT é basicamente formada de camadas de codificador empilhadas. Por exemplo, o modelo BERT base é constituído de 12 camadas de codificador, com camada oculta de tamanho 768 e com 12 *Attention-heads* (Devlin et al., 2019).

Das arquiteturas atuais, o modelo BERT é o modelo que vem aprimorando o estado-da-arte no PLN. Em avaliação de desempenho realizada através do benchmark GLUE (Wang et al., 2018), através de um conjunto de 9 tarefas diversas de compreensão de linguagem natural, o BERT aprimorou substancialmente a precisão do estado da arte em todas elas (Devlin et al., 2019). Onde: CoLa (*The Corpus of Linguistic Acceptability*) é utilizado para a tarefa de Aceitação Linguística; SST-2 (*The Stanford Sentiment Treebank*) é utilizado para a tarefa de Análise de Sentimento; MPRC (*Microsoft Research Paraphrase Corpus*) é utilizado para a tarefa de Paráfrase de Pesquisa; STS-B (*Semantic Textual Similarity Benchmark*) é utilizado para a tarefa de Similaridade Semântica; QQP (*Quora Question Pairs*) é utilizado para a tarefa de Pares de



Perguntas; MNLI-m (*The Multi-Genre Natural Language Inference*) é utilizado para a tarefa de Classificação de Múltiplos Gêneros; QNLI (*Question Natural Language Inference*) é utilizada para a tarefa de Perguntas e Respostas; RTE (*Recognizing Textual Entailment*) é utilizada para as tarefas de Perguntas e Respostas, Recuperação de Informação, Extração de Informação e Sumarização.

2.5 BERTIMBAU

O BERTimbau é uma versão do modelo BERT pré-treinado em corpora da língua portuguesa. Esse modelo foi pré-treinado para os dois tamanhos de modelo definido no BERT original: BERT-base cased (12 *self-attention layers*, 12 *attention-heads* e 768 *hidden size*) e BERT-large cased (24 *self-attention layers*, 16 *attention-heads* e 1024 *hidden size*), os dois com 340M *parameters*. Para esse modelo, o tamanho máximo de uma sentença é de 512 tokens (Souza et al., 2019).

Os experimentos conduzidos por Souza et al. (2019) foram realizados na tarefa de REN. Dessa forma, duas abordagens de transferência de aprendizado foram avaliadas: *feature-based* e *fine-tuning*. Em ambas as abordagens são realizados experimentos com e sem CRF (do inglês, *Conditional Random Fields*) *layer*.

Para a abordagem de *feature-based*, os pesos do BERT são mantidos e apenas o modelo do classificador e a CRF *layer* são treinadas. Já para a abordagem *fine-tuning*, o classificador é uma linear *layer* e todos os pesos são atualizando durante o treinamento Souza et al. (2019).

Na fase de pré-treinamento, Souza et al. (2019) utilizou o corpus brWaC contendo 2,68 bilhões de tokens providos de 3,53 milhões de documentos. Nesse processo, além de utilizar o corpo do documento (ignorando os títulos), o corpus foi pós-processado para corrigir tokens que foram decodificados utilizando a codificação incorreta. Por exemplo, a palavra “codificação” torna-se “codifica ããã £ o” quando codificada com UTF-8 e decodificada usando ISO-8859-1 (Souza et al., 2019).

Nas fases de treinamento e teste, foram utilizadas as CD do Primeiro HAREM e do Mini HA-REM, respectivamente. Os experimentos foram conduzidos em dois cenários: um cenário considerando 5 classes de entidades (Pessoa, Organização, Local, Valor e Tempo) e um cenário considerando as 10 classes de entidades utilizadas no HAREM (Abstração, Acontecimento, Coisa, Local, Obra, Pessoa, Organização, Tempo, Valor e Outro) (Souza et al., 2019).



Contudo, para que as CD do HAREM estivessem aptas para as fases de treinamento e teste do modelo, foi necessário realizar um pré-processamento no corpus do HAREM. Esse passo foi motivado devido às coleções do Primeiro HA-REM e do Mini HAREM considerarem a imprecisão e indeterminação do texto, como ambiguidade nas frases. Assim, através da tag <ALT> em alguns segmentos do corpus, foram incluídas várias soluções alternativas de identificação de EN. Em outras palavras, várias categorias puderam e foram atribuídas a uma mesma EN. Sabendo disso, Souza et al. (2019) pré-processaram as coleções de forma a selecionar a alternativa que continha o maior número de ENs marcadas. Caso houvesse empate, a primeira marcação era selecionada (Souza et al., 2019).

2.6 AVALIAÇÃO DE DESEMPENHO

As avaliações de desempenho de modelos de *Machine Learning* e *Deep Learning* são geralmente baseadas em métricas clássicas de Precisão, Revocação, Acurácia e Medida-F. De tal forma, olhar para essas métricas não responde como, porque e o que está influenciando no resultado gerado por um modelo em questão (Fu et al., 2020).

2.7 TRABALHO RELACIONADO

Na literatura não existem muitos trabalhos que realizam a análise dos resultados de modelos BERT ajustados para a tarefa de REN de forma sistemática. O trabalho que mais se aproxima ao nosso trabalho é o trabalho de Ichihara et al. (2015). Ele apresenta uma análise de baixa granularidade dos resultados do analisador de dependência japonesa e estrutura de caso chamado de KNP (*Kurohashi Nagao Parser*) em REN no BCCWJ (*Balanced Corpus of Contemporary Written Japanese*).

Operando como um analisador de estrutura de dependência baseado na gramática de dependência, o KNP utiliza um dicionário que possui uma ampla cobertura sobre informações sintáticas e regras heurísticas da língua Japonesa.

Com o objetivo de promover uma análise de erro do KNP em seis gêneros de texto para revelar as causas dos erros do analisador KNP e auxiliar no seu aperfeiçoamento, Ichihara et al. (2015) desenvolveu sua metodologia de análise em duas bases: (1) Métricas de desempenho para avaliação: Precisão, Revocação, Acurácia e Medida-F. Onde a anotação de token (sendo que uma categoria de entidade pode conter mais de uma palavra) só é considerada correta se a tag e o seu



intervalo estiverem corretos. (2) Erros do anotador organizados em uma das 5 seguintes categorias: sem extração, sem anotação, erro de faixa, tag errada, faixa e tag erradas. Onde: **sem extração** é o erro relacionado a não anotação do token, embora tenha sido anotado como EN no corpus; **sem anotação** é o erro relacionado a anotação do token, embora não tenha sido anotado como uma EN no corpus; **erro de faixa** é o erro na anotação parcial do token, ou seja, a anotação não iniciou ou terminou na palavra que representa início ou fim do token; **tag errada** é o erro relacionado a anotação do token como sendo uma EN diferente da anotada no corpus; **faixa e tag erradas** é erro onde o modelo anotou o token como sendo outra EN e com intervalo diferente do que foi anotado no corpus.

Estabelecido as bases, a abordagem foi dividida em duas partes, denominadas: análise de erro do KNP no BCCWJ e análise da categoria de erro com maior taxa de erro.

Na primeira parte, as métricas de desempenho do anotador e a sumarização dos erros por categoria foram apresentadas. A partir da sumarização, os autores identificaram a categoria de erro mais frequente na utilização do KNP e, posteriormente, o quanto essa categoria representava do total de erros em cada tipo de EN. Utilizando a métrica de acurácia por EN e a taxa de erro dessa categoria por tipo de EN. Ainda, neste trabalho foi identificada a EN onde está concentrada a maior parte dos erros da categoria e correlacionaram os resultados encontrados com as métricas de desempenho do anotador.

Na segunda parte, o processo de sumarização dos erros por gêneros de texto e o quanto a categoria do erro que ocorre com maior frequência na utilização do KNP representava do total de erros em cada gênero foram realizadas. Através desse processo é identificado o gênero de texto cuja taxa de erro dessa categoria é a mais alta e o gênero cuja acurácia é a maior. Partindo desses resultados, são apresentados exemplos de erros que ocorrem nos documentos dos gêneros de texto selecionados.

Através dos exemplos, Ichihara et al. (2015) levantam algumas hipóteses do porque do anotador ter um desempenho inferior em um determinado gênero de texto. Como principais, baseados nas características dos gêneros de texto selecionados e no dicionário que o KNP usa, listam problemas como o tipo da escrita (escrita usual geram análises morfológicas e sintáticas equivocadas) e a palavras novas nunca vistas antes pelo anota-dor. Da mesma forma, expõem possíveis soluções como: treinamento do anotador com textos informais e aumento do dicionário do anotador através da Wikipédia.



3 METODOLOGIA

Nesta seção é apresentada a metodologia de avaliação do modelo BERTimbau aplicado a tarefa de REN em Português. Após o estudo da literatura, examinamos os pontos que influenciaram no desempenho da tarefa de REN em Português para o corpus MiniHAREM, com a finalidade de identificar as características que podem gerar ou não a assertividade na classificação da EN. Por fim, agrupamos os padrões encontrados nos textos de entrada onde o modelo falha na classificação por classe da tarefa de REN (Local, Organização, Pessoa, Tempo e Valor).

Nós dividimos o desenvolvimento do trabalho em duas etapas. Na primeira etapa nos definimos o grupo de classes e o modelo BERTimbau a ser analisado e a segunda etapa nós realizamos a análise dos resultados, segmentada em 4 partes da seguinte forma: (1) **Amostragem de dados estatísticos** - realizamos esse processo para extrair as informações relacionadas a classificação do modelo, como as métricas clássicas e o total de tokens identificados no corpus. Essas informações serão utilizadas como base nas próximas partes. (2) **Escolha da classe com menor precisão** - através da base estatística levantada, inicializamos a análise dos erros de classificação através da classe com menor precisão. A escolha inicial da classe de menor precisão é devido a possível maior abrangência de padrões de erros que poderão ser utilizados como base na análise das classes seguintes. (3) **Análise individual do erro** - com a classe definida, extraímos todas as classificações incorretas do modelo e realizamos uma análise individual sobre cada uma delas. Nesse processo, identificamos os possíveis problemas que dificultaram a classificação correta do modelo. (4) **Agrupamento dos erros** - com todos as classificações incorretas identificadas e analisadas, realizamos o agrupamento do erros por semelhança/padrão, identificando assim um ponto fraco do modelo.

A segunda etapa possui uma configuração específica para a análise do erro de classificação. Nela foi definido que a análise de um token deve ser realizada em sentenças com 80 palavras a direita e 80 palavras a esquerda do token, ou seja, sentenças de pelo menos 160 palavras.

Com o tamanho da sentença definida e, posteriormente, com a sentença extraída, primeiramente é realizada a análise individual do token, sem a análise contextual. Utilizando esse processo extraímos informações como: a classificação do token iniciou na palavra errada, a



classificação terminou na palavra errada ou a classificação iniciou e terminou na palavra certa porém foi classificado na classe errada.

Esse processo é possível pois os tokens anotados no corpus e classificados pelo modelo estão no formato IOB (Li et al., 2022)(do inglês, short for *Inside, Outside, Beginning*). Esse formato é utilizado para anotar tokens em tarefas de agrupamento, como por exemplo a tarefa de REN. Através desse formato, é possível obter a informação sobre a localização das palavras no token devido a seguinte marcação: B-{categoria}: identifica a palavra que inicia o token; I-{categoria}: identifica a/as palavra/palavras que fazem parte do token; O - palavras que não fazem parte de nenhum token.

Com essa análise inicial, conseguiremos listar problemas, por exemplo, na classificação de prefixos ou na utilização de caracteres especiais. Na sequência do segmento, realizamos uma análise contextual para verificação de ambiguidade dos tokens e a identificação e análise das estruturas da sentença que influenciaram na classificação do modelo.

Extraídas essas informações, analisamos a sua contabilização no corpus e extraímos em quais categorias o token já foi classificado. A partir disso, extraímos uma sentença de cada categoria que contém a anotação do token. Essas sentenças nos informarão se o modelo acabou decorando o token caso, em qualquer sentença, independente do contexto, seja anotada pela mesma classe. Por fim, levantamos todas as análises e definimos o motivo do erro naquela classificação.

4 DISCUSSÃO DOS RESULTADOS

Nesta seção são apresentadas as análises realizadas sobre o grupo de classes e o modelo BERTim-bau, as estatísticas do corpus, análise individual dos erros e o agrupamento dos erros.

4.1 GRUPO DE CLASSES E MODELO BERTIMBAU

Definimos o corpus do MiniHarem com 5 classes para realizar os testes no modelo BERTimbau. O motivo da escolha desse *dataset* foi o número total de ENs anotadas manualmente (3.758). O grupo de classes avaliadas foram Local, Pessoa, Organização, Valor e Tempo. Já que essas classes são as mais frequentes no *dataset* 895, 836, 622, 364 e 328 ocorrências, respectivamente. Somente os seguintes checkpoints foram disponibilizados no



GitHub da NeuralMind-AI¹: BERTimbau base cased - CRF (selective scenario, 5 classes) e BERTimbau base cased (total scenario, 10 classes).

Dessa forma, como já havíamos definido a utilização do corpus do MiniHarem, o modelo utilizado nos experimentos foi BERTimbau base cased - CRF. Ele é composto por 12 self-attention layers, 12 attention-heads e 768 hidden size. O modelo foi treinado com 340 milhões de parâmetros e com sentenças de tamanho máximo de 512 palavras.

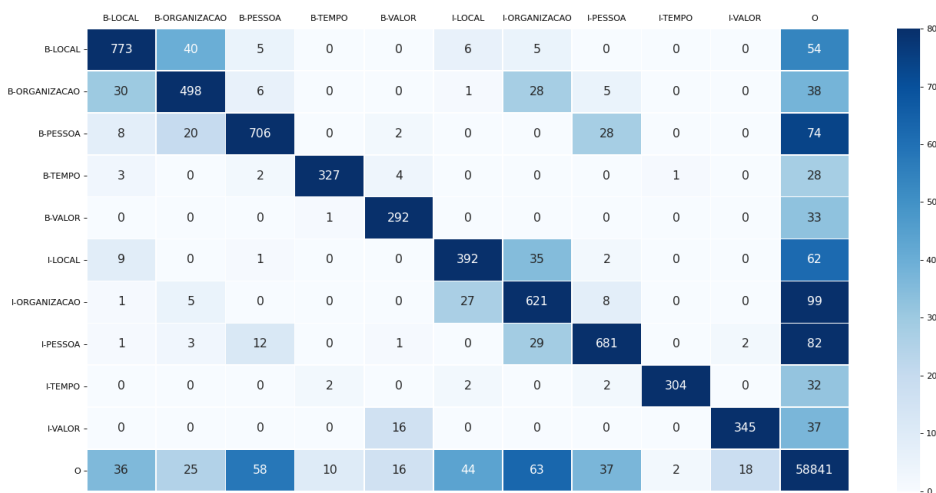
4.2 AMOSTRAGEM DE DADOS ESTATÍSTICOS

Nas Tabelas 1, 2 e 3 são apresentadas, respectivamente, as estatísticas do corpus, as métricas clássicas gerais do modelo e as métricas clássicas do modelo por classe. Além disso, podemos visualizar o total de tokens anotados no corpus (2.489) e o total de tokens identificados pelo modelo (2.913). É importante mencionar que alguns tokens não foram anotados e deveriam ser, por esse motivo, o total de tokens identificados é maior que o total de tokens anotados.

Para que pudéssemos nos aprofundar nos resultados alcançados nas Tabelas 2 e 3, realizamos a amostragem das frequências de classificação para cada classe do modelo através de uma matriz de confusão. A matriz de confusão apresentada na Figura 1 expõe informações importantes sobre a classificação do modelo, como: o total de tokens anotados no corpus por classe, total tokens identificados pelo modelo por classe, total de tokens por classe que foram classificados incorretamente em uma das outras classes e o total de tokens por classe que iniciaram na palavra errada.



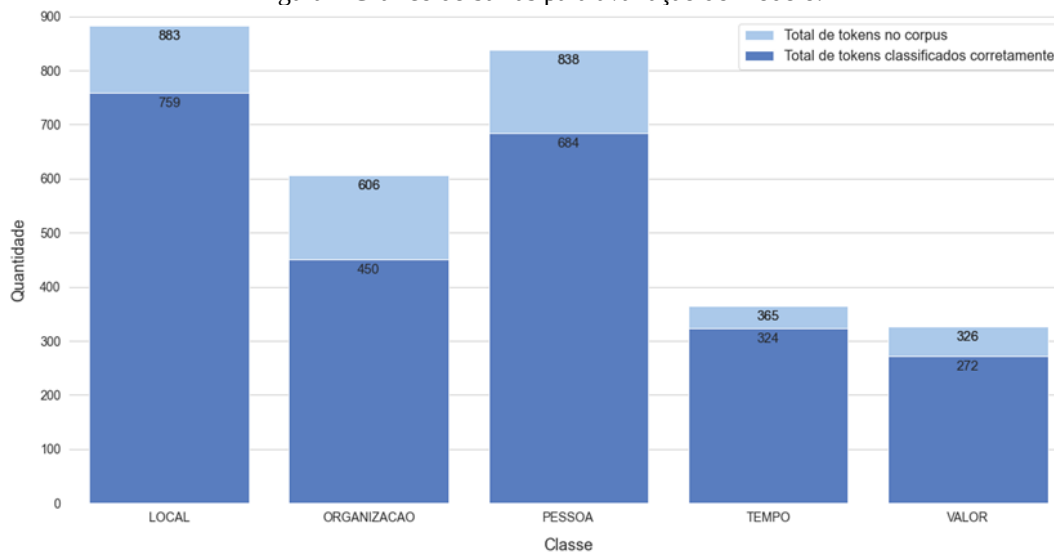
Figura 1 Matriz de confusão dos resultados gerados pelo modelo REN baseado no BERTimbau e treinado para o MiniHAREM.



Fonte: Própria.

Na Figura 2 temos o gráfico de barras representando o total de tokens classificados corretamente pelo modelo, e o total de tokens anotados no corpus. Correlacionando a diferença entre o total de tokens classificados corretamente e o total de tokens com classificação iniciada na palavra correta, que pode ser extraído pela matriz de confusão, nós obtemos a informação do total de tokens que foram classificados incorretamente por terminarem na palavra errada.

Figura 2 Gráfico de barras para avaliação do modelo.



Fonte: Própria.



4.3 ANÁLISE INDIVIDUAL DOS ERROS

Com os resultados obtidos através das métricas clássicas, conseguimos definir a classe para inicializar a avaliação individual dos erros do modelo (Tabela 3).

Para realizar o processo de avaliação, inicialmente extraímos cada um dos erros de classificação do modelo e realizamos uma análise individual de cada um deles, sem levar em conta, inicialmente, a contextualização da sentença.

Por exemplo, na Figura 3, notamos que a classificação do modelo iniciou e terminou na palavra correta, porém o modelo classificou a entidade na categoria Pessoa. Dessa forma, seguimos para a etapa de avaliação contextual do token para identificar na estrutura da sentença meios que podem ter influenciado na classificação incorreta do token.

Figura 3 Sentença com uma entidade classificada incorretamente pelo modelo.

Comecei a gostar dos computadores, apegou-se o bichinho e comecei a aperfeiçoar-me melhor. Tirei o curso de formação, no Centro de Formação de Informático do Minho. E nessa altura já trabalhava na universidade? Não , ainda não estava a trabalhar na Universidade do Minho, estava a estudar, Alberto **B-ORGANIZACAO | B-PESSOA** Sampaio **I-ORGANIZACAO | I-PESSOA** e ia concorrendo. Cada vez que saíam concursos nos jornais para a função pública, eu fazia uma candidatura e mandava. Ia fazendo o curso aumentava o currículo e mandava. Ia mandando, até que uma vez deu-me para ir às provas e fiquei. E daí ficou efectiva na Universidade? Efectiva,'.

Fonte: Própria.

Podemos notar que a possível falta de um conector entre a frase ‘estava a estudar’ e o nome da instituição ‘Alberto Sampaio’, que resultou em uma quebra de contexto, pode ter auxiliado o modelo a não identificação de ‘Alberto Sampaio’ como o nome de uma organização.

Contudo, ainda não podemos ter certeza se o modelo errou por conta da quebra contextual, ambiguidade do token ou se ele apenas decorou o token como Pessoa. Assim, verificamos se o modelo decorou ou não esse token analisando a sua anotação em sentenças do corpus com contextos diferentes. A partir dessas informações então, conseguimos definir o erro de classificação do token e reiniciar o processo para o próximo token.

Nesse caso, após levantadas todas essas informações, podemos concluir que essa classificação incorreta ocorreu por um problema contextual em conjunto com a ambiguidade do token, sendo assim anotado como Organização e classificado como Pessoa.



Tabela 1: Estatísticas do corpus.

Modelo	Total de Palavras	Total de Tokens	Total de Palavras Distintas	Total de Classes
BERTimbau-CRF base	64.981	2.489	12.610	5

Fonte: Própria.

Tabela 2: Resultado do modelo: Métricas Clássicas Média Micro.

Modelo	Acurácia	Acurácia (non O)	Precisão	Revocação	Medida-F	Total de tokens identificados
BERTimbau-CRF base	98,15%	84,70%	85,45%	82,47%	83,93%	2.913

Fonte: Própria.

Tabela 3: Resultado do modelo por classe: Métricas Clássicas.

Classe	Precisão	Revocação	Medida-F	Total de tokens identificados
LOCAL	88,15%	85,96%	87,04%	861
ORGANIZAÇÃO	76,14%	74,26%	75,19%	591
PESSOA	86,58%	81,62%	84,03%	790

Fonte: Própria.

4.4 AGRUPAMENTO DOS ERROS

Nessa subseção serão apresentados todos os agrupamentos de erro de classificação do modelo, segmentados por classe.

4.4.1 Organização

Através da abordagem inicial de avaliação dos resultados do modelo, geramos 7 agrupamentos com base em padrões de erro na classificação dos tokens relativos à classe **Organização**. Eles são apresentados a seguir.

- Agrupamento 1: Tokens que representam o nome de uma organização e aparecem apenas uma vez na sentença/corpus ou em sentenças sem contexto

No corpus, 147 tokens foram mencionados e anotados como **Organização** apenas uma vez. Dos quais, alcançando uma precisão de 76,19%, o modelo classificou corretamente 112 deles. Dos 35 tokens classificados incorretamente pelo modelo, 4 foram classificados como **Organização** e iniciados na palavra errada, 14 como **Outros**, 11 como **Local** e 6 como **Pessoa**.

- Agrupamento 2 - Substantivo que remete a uma ou várias organizações

A partir de substantivos que podem remeter a uma ou várias organizações, como governo, república, estado, público, escola, associação, união e senado, extraímos do corpus 51 tokens anotados como **Organização**. Desses 51 tokens, o modelo classificou corretamente 38. Dos 13



tokens classificados de forma incorreta pelo modelo, 2 foram classificadas como **Pessoa**, 4 como **Local** e 7 como **Outros**.

- Agrupamento 3 - Nome de organização ou substantivo que remete a uma organização entre caracteres especiais

Para realizar a análise desse agrupamento, retiramos amostras de tokens que se encontravam entre aspas duplas, aspas angulares e parênteses.

Dessa forma, foram encontrados no corpus 4 tokens anotados como **Organização** entre as aspas duplas, dos quais, o modelo classificou corretamente 3 deles, ou, 75% dos tokens. No caso das aspas angulares, foram encontrados no corpus 13 tokens anotados como **Organização** entre elas, dos quais, o modelo classificou corretamente 6, ou, 46,15% dos tokens. E no caso dos parênteses, foram encontrados no corpus 40 tokens anotados como **Organização**. Desses, o modelo classificou corretamente 32, ou, 80% dos tokens. De forma geral, foram classificados corretamente 71,92% dos tokens que se encontravam entre esses caracteres especiais.

No caso dos tokens anotados entre aspas duplas, normalmente esses caracteres especiais não interrompem o contexto/fluxo da frase – os tokens não são colocados fora do contexto da sentença –, apenas aparecem para destacar o texto. No caso dos tokens anotados entre aspas angulares, eles também não são posicionados de forma a interromper o contexto/fluxo da frase. Esses caracteres especiais são normalmente utilizados em transcrições, citações e frases e/ou expressões em análise.

No caso dos tokens anotados entre parênteses, o modelo classificou corretamente todos os casos que um token se referia a sigla do nome de uma **Organização**. Contudo, encontramos 2 casos que esse padrão de token não é anotado no corpus, mas é classificado pelo modelo como **Organização**.

- Agrupamento 4 - Nome da organização e o local de sua sede separados por um caractere especial

Nesse caso, todos os tokens anotados como **Organização** no corpus, estando eles separados por uma barra (/) de outro token anotado como **Organização**, o modelo não identifica como duas organizações distintas. Em sentenças que apresentam esse padrão de tokens, o modelo classifica como se os dois tokens fossem a mesma **Organização**.



O corpus apresenta 3 casos onde “CPDA/UFRRJ” não é anotado como **Organização/Local** pelo corpus. É incongruente, já que em outras 15 ocasiões essa mesma sentença foi anotada pelo corpus como **Organização/Local**.

- Agrupamento 5 - Nome da organização igual ao nome de pessoa

Foram encontrados 14 tokens anotados como **Organização** que eram compostos por um nome de uma pessoa já citada e anotada anteriormente no corpus como **Pessoa**. Dessas, o modelo classificou incorretamente 3 tokens como **Pessoa**, totalizando 78,57% de acerto. Nesse padrão, as sentenças que iniciam indicando o seguimento da instituição, por exemplo “Escola Miguel Viera”, o modelo apresenta 100% de acerto (8 tokens). Contudo, casos onde a classificação depende do contexto, o modelo não obtém o mesmo resultado, já que conseguiu classificar corretamente apenas 50% dos tokens (6 tokens).

- Agrupamento 6 - Nome da organização seguida pelo nome de uma pessoa e vice-versa

Foram encontradas 7 sentenças que continham um token da classe **Organização** seguido por um token da classe **Pessoa** e vice-versa. Os tokens foram classificados corretamente pelo modelo em apenas 3 sentenças (42,85% de acerto). Nas sentenças em que o modelo acerta a classificação, o token da classe **Pessoa** é seguido pela sigla de uma **Organização** já citada anteriormente.

- Agrupamento 7 - Tokens que representam duas organizações, ou uma organização e uma pessoa, separadas por um palavra que indica posse, ou tokens que representam uma organização e de um substantivo que remeta a tempo, local ou valor, separados por da/das/do/dos/de.

Nesse padrão, encontrou-se no corpus 9 sentenças, das quais, o modelo classificou corretamente 3 tokens. Notamos que, quando o nome de uma **Organização** é seguido de um substantivo que remete **Tempo** ou **Valor**, o modelo classifica corretamente todos esses casos. Contudo, quanto o nome de uma **Organização** é seguido por um substantivo anotado como **Local**, **Pessoa** ou **Organização**, o modelo classifica como se os tokens fizessem parte de um único token da entidade **Organização**.

Foi considerado também a amostragem de tokens anotados como **Organização** que continham em sua estrutura da/das/do/dos/de. Com essa configuração, foram encontrados no



corpus 163 tokens. Dessas, 142 foram anotados de forma correta pelo modelo, com, aproximadamente, 87,11% de acerto.

4.4.2 Valor

Através da abordagem inicial de avaliação dos resultados do modelo, geramos 5 agrupamentos com base em padrões de erro na classificação dos tokens relativos à classe **Valor**. Eles são apresentados a seguir.

- Agrupamento 1 - Número descrito no seu formato extenso

No corpus, foram encontrados 6 tokens que representam um número por extenso, que são: Dez, Trinta, Dois, Cinco, Três e Um. Dessas 6 anotações, em todos os casos o modelo classificou incorretamente como **Outros**.

- Agrupamento 2 - Número romano

No corpus, encontramos 21 números romanos, dos quais, apenas 3 foram anotados no corpus como **Valor**. Já para o modelo, nenhum número romano é classificado como **Valor**.

Em 11 tokens, os números romanos são anotados no corpus como **Outros**. Sendo que, em 8 tokens, os números romanos são mencionados em nomes de eventos para indicar o grau de acontecimento do evento. Nas outras 6 ocorrências de número romano, eles são utilizados para representar um século em específico. Para esse caso, tanto o corpus quanto o modelo classifica como **Tempo**. Por fim, um caso é encontrando fazendo menção ao nome de uma instituição. Tanto o corpus quanto o modelo classifica como **Organização**.

- Agrupamento 3 - Prefixo que acompanha valores numéricos

Foram coletadas amostras através das seguintes palavras: aproximadamente, quase, menos, cerca e mais. No total foram encontrados em 32 sentenças, prefixos acompanhando um **Valor** numérico no corpus. Dessas, o modelo classificou corretamente em 17 sentenças, ou seja, aproximadamente 53,12% dos tokens foram classificados corretamente.

Das palavras buscadas no corpus: (i) a palavra ‘aproximadamente’ apareceu em 3 tokens como **Valor** e foi classificado corretamente em apenas um deles; (ii) a palavra ‘quase’ apareceu em 2 tokens como **Valor**, com uma classificação correta; (iii) a palavra ‘menos’ apareceu uma vez como **Valor** e foi classificada incorretamente; (iv) a palavra ‘cerca’ apareceu em 12 tokens como **Valor** e foi classificada em 11 delas de forma errada; (v) a palavra ‘mais’ apareceu em 15 tokens, dos quais o modelo acertou todos.



Tendo em vista que 88,24% das classificações corretas vem dos tokens que contem a palavra ‘mais’, ao retirar os tokens que contêm essa palavra, o modelo classifica corretamente apenas 11,76% dos tokens.

- Agrupamento 4 - Caracteres ° e ª que acompanham valores numéricos

No total foram encontrados 22 tokens anotados como **Valor** que contem o carácter ° ou ª. Mais precisamente, 19 tokens contem o carácter °, dos quais o modelo classifica corretamente 16 deles, e 3 tokens contendo ª, sendo eles classificados corretamente. Nesse caso, o modelo teve 86,36% de classificação correta.

- Agrupamento 5 - Unidade de medida que acompanha um valor numérico

Nesse agrupamento está pelo menos 80% dos erros de classificação do modelo relacionados ao tamanho do token. Foram encontrados pelo menos 162 tokens no corpus que possuem um número acompanhado da sua unidade de medida. Dentre as unidades de medida encontradas, são elas: metros, euros, reais, por cento, dias, minutos e horas. Dos 162 tokens, o modelo classifica incorretamente a unidade de medida em 16 tokens, ou seja, cerca de 9,12% dos tokens classificados corretamente. Contudo, o problema nesse padrão está na identificação das unidades de medida. Em muitas sentenças, valores numéricos são utilizados para informar a quantidade de um determinado objeto, como, por exemplo, fotos. Seguindo essa ideia, alguns dos substantivos que acompanham esses valores são classificados como parte do token de forma errada pelo modelo.

4.4.3 Pessoa

Através da abordagem inicial de avaliação dos resultados do modelo, geramos 3 agrupamentos com base em padrões de erro na classificação dos tokens relativos à classe **Pessoa**. Eles são apresentados a seguir.

- Agrupamento 1 - Prefixo antes de nome de pessoas/citação de pessoa

Foram coletadas amostras através das seguintes palavras: senhor, senhora, avô, avó, irmão, irmã, irmãos, tio e mana.

No corpus, a palavra senhora antecede 17 no mês citados no texto. Desses, 9 são escritos com a letra inicial em maiúsculo e o modelo acerta em todos os casos. Contudo, em 8 casos ela aparece iniciada em letra minúscula, das quais o modelo marca como **Pessoa**. Existe um caso



onde senhora é citado, em letra minúscula, no meio de um token de **Pessoa**. O modelo não identifica esse token.

No corpus, a palavra senhor antecede 17 no mês citados no texto. Desses, 7 são escritos com a letra inicial em maiúsculo e o modelo acerta em todos os casos. Contudo, em 10 casos ela aparece iniciada em letra minúscula, das quais o modelo marca como **Pessoa**.

No corpus, a palavra tio antecede 14 nomes citados no texto. Desses, 5 são escritos com a letra inicial em maiúsculo e o modelo acerta em todos os casos. Contudo, em 8 casos ela aparece iniciada em letra minúscula, das quais o modelo marca 7 como **Pessoa**. Dessas 8, o corpus anota todas como **Pessoa**. Existe um caso onde tio é citado, todo em letra maiúscula, e é anotado como **Pessoa** pelo corpus e **Outros** pelo modelo.

- Agrupamento 2 - Substantivo que remete a uma ou a um grupo de pessoas

A ambiguidade gerada pela citação do nome de organizações é o principal problema nesse caso. Muitas vezes o nome de uma **Organização** está sendo citado com o intuito de expor os colaboradores daquela **Organização** e não para falar da **Organização** em si, o que gera confusão na classificação dos tokens. Outro padrão de erro encontrado dá-se no exemplo da seguinte sentença: “Diretores: José Angelo”. O corpus anota ‘Diretores’ como **Pessoa**, porém o modelo não entende dessa forma e classifica ‘Diretores’ como **Outros**. No corpus encontramos sentenças com as seguintes palavras: Orientador, Professor, Árbitros, Direção, Coordenador, Debatedores e Professora. Essa busca resultou em 11 sentenças, das quais o modelo classificou corretamente apenas uma.

- Agrupamento 3 - Nome de pessoa citado como resposta a uma pergunta

Em textos do gênero Perguntas e Respostas no corpus, encontramos 12 sentenças em que é realizado perguntas em relação ao nome de uma **Pessoa**. Identificamos que nessas 12 sentenças, não importa se era para saber o nome da **Pessoa** perguntada, dos pais, avôs, ou de conhecidos da **Pessoa**, o modelo não interpretou, naquelas repostas, o nome respondido como uma entidade do tipo **Pessoa**. Das 12 sentenças encontrados no corpus, há 6 casos em que 1) o nome vem logo em seguida da pergunta e há outros 6 casos em que 2) o nome vem no meio da resposta.



4.4.4 Local

Através da abordagem inicial de avaliação dos resultados do modelo, geramos 5 agrupamentos com base em padrões de erro na classificação dos tokens relativos à classe **Local**. Eles são apresentados a seguir.

- Agrupamento 1 - Nome de um Local igual a nome de uma Pessoa

Normalmente o modelo erra por dois motivos: 1) quando a palavra aparece mais de uma vez e é classificada muitas vezes como **Local** ou como **Pessoa** e 2) por contas dos prefixos, quando está se referindo a um objeto de alguém, por exemplo.

A palavra ‘Campos’, por exemplo, aparece 3 vezes no corpus em 3 sentenças diferentes. Das 3 sentenças, duas delas são anotadas como sendo sobrenome de uma **Pessoa**, tanto pelo corpus quanto pelo modelo. A terceira aparece como nome de um **Local**, o modelo prevê como **Pessoa**.

A palavra ‘Miranda’, por exemplo, aparece 3 vezes no corpus em 3 sentenças diferentes. Das 3 sentenças, duas são anotadas como sendo um **Local**, tanto pelo corpus quanto pelo modelo. A terceira aparece como nome de uma **Pessoa**, o modelo prevê como **Local**.

- Agrupamento 2 - Nomes de locais seguidos sem nenhum separador

Esse padrão é difícil até para um anotador humano. Esse padrão ocorreu em textos extraídos da internet e jornais, onde tabelas, links, atalhos para outras páginas são distribuídos no texto de forma desestruturada. No corpus, em relação a nome de locais, somente houve 3 casos, dos quais, o modelo classificou incorretamente.

- Agrupamento 3 - Nome de dois lugares ou de um lugar e uma Organização, separados por caracteres especiais

Nesse padrão, todas as sentenças que possuem dois tokens, separados por uma barra e que representam dois Lugares ou uma **Organização** e um **Lugar**, o modelo os classifica como se fossem apenas um **Local**.

- Agrupamento 4 - Menção do nome da Organização como forma de falar do Local de sua sede

Os casos mais frequentes desse padrão erro do modelo estão em 1) citações no intuito de falar do **Local** de um país, 2) em citações no intuito de falar da sede de uma **Organização**, 3) textos retirados da web, quando se menciona a página/site de uma **Organização** e 4) em sentenças com palavras ambíguas.



A palavra Brasil, por exemplo, é citada em 90 sentenças no corpus, sendo em só uma como **Organização**. Nessa sentença o modelo a classifica incorretamente como **Local**. Das outras 89 sentenças, em 3 delas a palavra é anotada como **Local** e o modelo a classifica incorretamente como **Organização**. A palavra Portugal, por exemplo, é citada em 21 sentenças, em nenhuma é anotado como **Organização**. Já para o modelo, em pelo menos 3 sentenças em que o corpus anota como **Local**, é classificada como **Organização**. A palavra EUA é citado em 12 sentenças, sendo em 3 como **Organização**. O modelo classifica incorretamente uma como **Local**.

- Agrupamento 5 - Número remetendo a um Local

No corpus, ocorreram 8 casos onde um número de uma agência, conta de banco, número da rua, linha de trem, são anotados pelo corpus como **Local** e, em todos os casos, o modelo os classifica incorretamente como **Outros**.

4.4.5 Tempo

Através da abordagem inicial de avaliação dos resultados do modelo, geramos 2 agrupamentos com base em padrões de erro na classificação dos tokens relativos à classe **Tempo**. Eles são apresentados a seguir.

- Agrupamento 1 - Substantivo comum que remete a um período ou a um acontecimento no passado

No corpus são anotados 88 tokens onde substantivo comum que remete a um período ou a um acontecimento no passado. Dos 88, o modelo classifica corretamente 57 token, cerca de 64,77% de acerto. A palavra Natal (referente ao nascimento de Jesus), por exemplo, aparece no corpus em 7 casos e é anotada corretamente como **Tempo**. Como nome de **Pessoa**, foram encontrados no corpus apenas 2 casos. Os dois casos o modelo errou.

- Agrupamento 2 - Falta de contexto

No corpus foi identificado 7 tokens inseridos em sentenças do texto sem nenhum contexto. O modelo erra os 7 tokens.



5 CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS

O objetivo geral desse trabalho foi apresentar uma análise dos resultados do modelo BERTimbau para a tarefa de REN no *dataset* MiniHA-REM. O intuito foi verificar os pontos fracos do modelo, seus padrões de erros e confirmar se o desempenho apresentado na tarefa REN é, de fato, justificado.

Através da análise realizada, o modelo BERT demonstrou se capaz de reconhecer entidades nomeadas. Nos agrupamentos apresentados, notamos que a grande parte dos erros estão ligados a ambiguidade de tokens, a tokens posicionados em sentenças sem contextualização, a tokens citados apenas uma vez no corpus posicionados em sentenças com pouca contextualização e padrões de tokens que possivelmente não estão no documento de treinamento, como número romanos, tokens entre caracteres especiais e caracteres especiais separados por dois tokens.

Conforme os agrupamentos mostrados e seus respectivos exemplos, notamos que a falta de estruturação de alguns textos, falta de alguns exemplos no documento de treinamento e a ambiguidade das palavras são os grandes motivos dos erros do modelo.

Como demonstrado nos agrupamentos, boa parte dos erros do modelo é devido à ambiguidade dos tokens. Como no corpus os tokens podem ser anotados com mais de uma entidade e pelo modelo permitir apenas uma entidade por token, não possuímos uma garantia que o modelo realmente errou em um dado token. Desse modo, é importante trabalhar em outra metodologia para garantir que o modelo está mesmo errando em casos de ambiguidade.

Alguns erros de classificação do modelo foram gerados em tokens distribuídos em sentenças sem contexto. Normalmente, esses casos ocorrem em textos retirados de páginas web, onde os títulos das páginas, atalhos para outras páginas, URL, rodapé, tabelas desconstruídas e cabeçalhos são distribuídos nas sentenças conforme o processamento realizado. Por conta disso, seria interessante realizar um pré-processamento no corpus.

Por fim, aplicar a técnica de *Data Augmentation* na base de treinamento do modelo para abranger os padrões que o modelo não aprendeu.



REFERÊNCIAS

- Devlin, Jacob, Ming-Wei Chang, Kenton Lee & Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. Em Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 4171–4186. Minneapolis, USA: Association for Computational Linguistics.
- Fu, Jinlan, Pengfei Liu & Graham Neubig. 2020. Interpretable multi-dataset evaluation for named entity recognition. Em Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 6058–6069. Online: Association for Computational Linguistics
- Henke, Nicolaus, J. Bughin, Michael K. F. Chui, James Manyika, Tamim Saleh & Bill Wiseman. 2016. The age of analytics: competing in a data-driven world. Relatório técnico. McKinsey Global Institute. <https://www.sipotra.it/wp-content/uploads/2017/01/THE-AGE-OF-ANALYTICS.pdf>.
- Ichihara, Masaaki, Tomoya Iwakura, Kanako Komiya & Maiko Yamazaki. 2015. Error analysis of named entity recognition in bc-cwj, 1–5. Online. https://www.anlp.jp/proceedings/annual_meeting/2015/html/paper/WS_PNN05_O1.pdf.
- do Lago Pereira, Silvio. 2011. Processamento de linguagem natural. <https://www.ime.usp.br/~slago/IA-pln.pdf>.
- Li, Jing, Aixin Sun, Jianglei Han & Chenliang Li. 2022. A survey on deep learning for named entity recognition. IEEE Transactions on Knowledge and Data Engineering 34(1). 50–70.
- Lopes, Maria Célia Santos. 2004. Mineração de dados textuais utilizando técnicas de clustering para o idioma português: Universidade Federal do Rio de Janeiro. Tese de Doutorado. <http://www.coc.ufrj.br/pt/teses-de-doutorado/148-2004/980-maria-celia-santos-lopes>.
- Mota, Cristina & Diana Santos (eds.). 2008. Desafios na avaliação conjunta do reconhecimento de entidades mencionadas: O segundo harem. <http://www.linguateca.pt/LivroSegundoHAREM/>.
- Nadeau, David & Satoshi Sekine. 2007. A survey of named entity recognition and classification. Lingvistica e Investigationes 30(1). 3–26.
- Nadkarni, Prakash M., Lucila Ohno-Machado & Wendy W. Chapman. 2011. Natural language processing: an introduction. Journal of the American Medical Informatics Association 18(5). 544–551.
- Santos, Diana & Nuno Cardoso (eds.). 2007. Livro do harem: Reconhecimento de entidades mencionadas em português. <https://www.linguateca.pt/Morfolimpiadas>.



e Silva, Andressa Vieira. 2020. Um modelo de classificação para o reconhecimento de entidades nomeadas: Universidade de São Paulo. Tese de Mestrado.

da Silva, Tiago Santos. 2012. Reconhecimento de entidades nomeadas em notícias de governo: Universidade Federal do Rio de Janeiro. Tese de Mestrado.

da Silva Magalhães Gomes, Diogo & Alexandre Gonçalves Evsukoff. 2019. Processamento de linguagem natural em português e aprendizagem profunda para o domínio de Óleo e gás. CoRR abs/1908.01674. 1–25.

Souza, Fábio, Rodrigo Frassetto Nogueira & Roberto de Alencar Lotufo. 2019. Portuguese named entity recognition using bert-crf. CoRR abs/1909.10649. 1–8.

Tukey, John W. 1962. The future of data analysis 408–452. Springer Series in Statistics. <http://www.jstor.org/stable/2237638>.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser & Illia Polosukhin. 2017. Attention is all you need. Em Proceedings of the 2017 Advances in Neural Information Processing Systems (NIPS), 5998–6008.

Wang, Alex, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy & Samuel Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. Em Proceedings of the 2018 Workshop Blackbox NLP: Analyzing and Interpreting Neural Networks for NLP (EMNLP), 353–355. Brussels, Belgium: Association for Computational Linguistics.